Syracuse University
School of Information Studies

Week 9
Supervised Machine Learning

Dr. Preeti Jagadev

Using Classification and Regression Trees

Reviewing The Modeling Process:

- **Use lots of training examples:**
  You need a big set of data to teach the machine learning algorithm.

- **Algorithm learns from that data:**
  The algorithm studies the training data and creates a “model”, basically a pattern or rule that helps it make predictions.

- **This is called supervised learning:**
  Because we give the algorithm both the input (data) and the correct answer (labels) during training.

- **Validate with new data:**
  This helps us see if the model is truly learning patterns, not just memorizing the training data.

- **We test it on new data:**
  We check how well the model works by using another set of data it hasn’t seen before, this is the **test data**.

- **Use it for predictions:**
  Once it works well, we can use the model to predict new or future data that wasn’t used in training.

Confusion Matrix

- A table that summarizes how successful a classification model is.
- This matrix has as many rows and columns as there is number of classes.
- ***x-axis:*** the label that the model predicts:

  y-axis: the actual label

  Diagonal elements: the predicted label is equal to the true label

  Off-diagonal elements: mislabeled by the classier.
- **The higher the diagonal values of the confusion matrix the better**

		Predicted class	
		P	N
Actual class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

Performance Metrics

The performance metrics are computed by using the formulas given below.

$$Accuracy(\%) = \frac{TP + TN}{TP + FP + TN + FN} \times 100$$

$$Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

$$Precision = \frac{TP}{TP + FP}$$

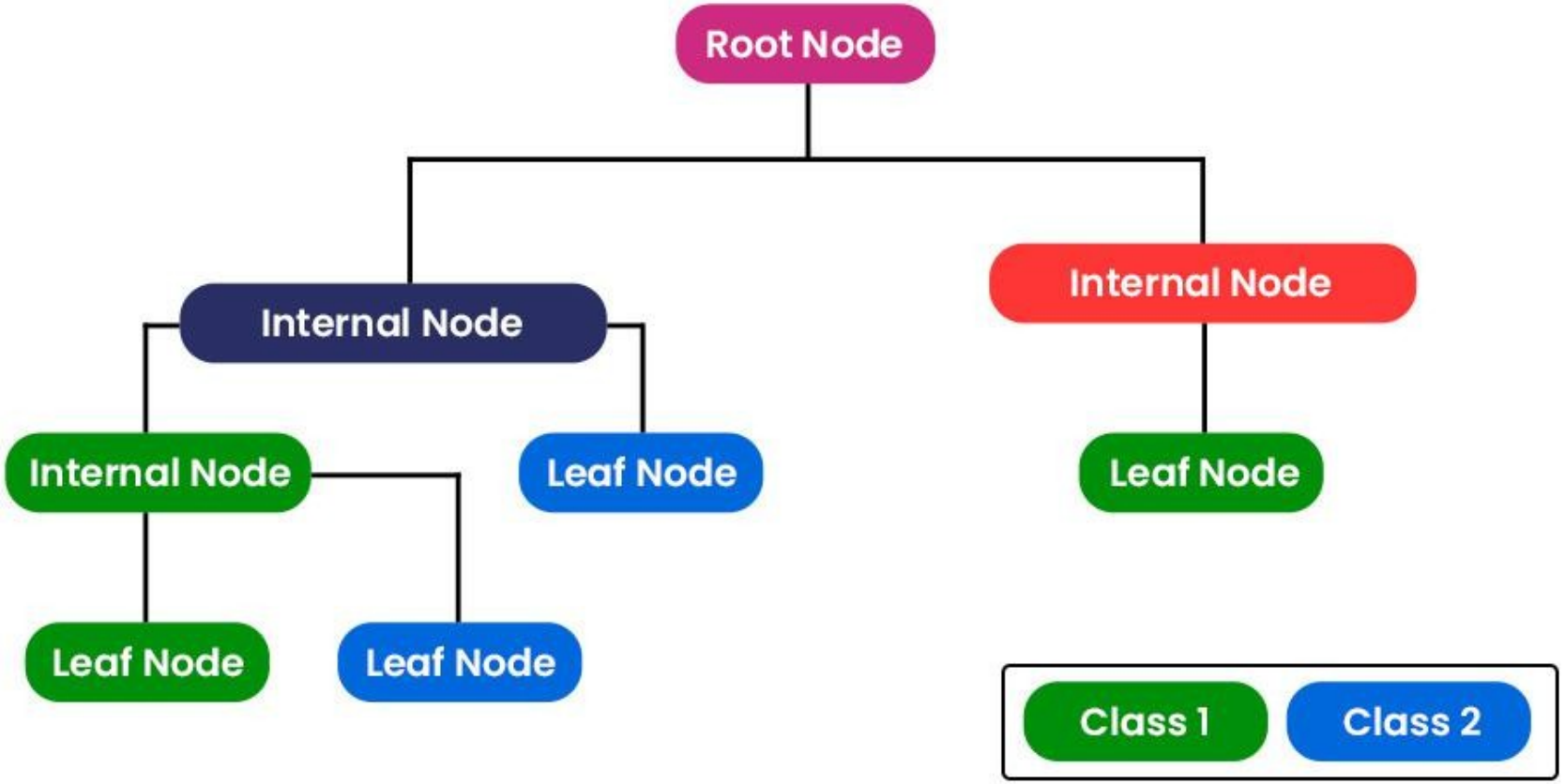
$$G - mean = \sqrt{Sensitivity \times Specificity}$$

$$F - Measure = 2 \times \frac{Precision \times Sensitivity}{Precision + Sensitivty}$$

Here ***TP, FP, TN***, and ***FN*** denote ***true positive, true negative, false positive***, and ***false negative,*** respectively.

K-Fold Cross-Validation

CART (Classification and Regression Tree)



CART (Classification and Regression Tree)

This picture shows how a CART works in **simple words**:

- **Start at the top (Root Node):**
  The algorithm starts with all the data at the top of the tree.

- **Split into smaller groups:**
  The root node divides the data based on the *best question* (attribute and threshold value), for example, “Is age > 30?”.

- **Internal Nodes:**
  These are the middle points where more splits happen. Each internal node checks another condition, further dividing the data into smaller and more similar groups.

- **Leaf Nodes:**
  These are the final points in the tree; they don’t split anymore.
  - Each leaf node represents a **final decision or prediction**, such as “Class 1” or “Class 2”.

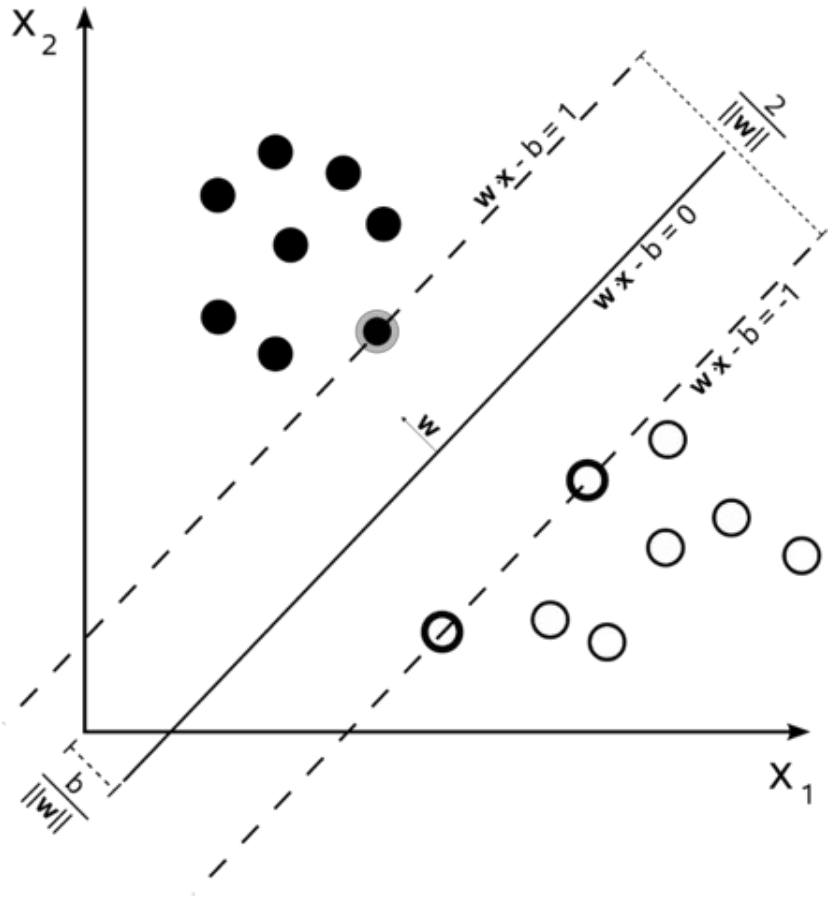
- **Repeat the process:**
  Every time the data is split, the algorithm applies the same logic again and again until it can’t split meaningfully anymore.

Support Vector Machines (SVM)

- This diagram illustrates the core idea behind a machine learning method called **Support Vector Machines (SVM).**
- The goal is to find the **best possible line (or boundary)** to separate two different groups of things (the black dots and the white circles).

The Key Concepts

- **The Groups (Dots and Circles):** These are your data points. The black dots belong to one category, and the white circles belong to another.
- **The Dividing Line :** This solid line is the **classification boundary**. It's the best separator the computer could find to put all black dots on one side and all white circles on the other.



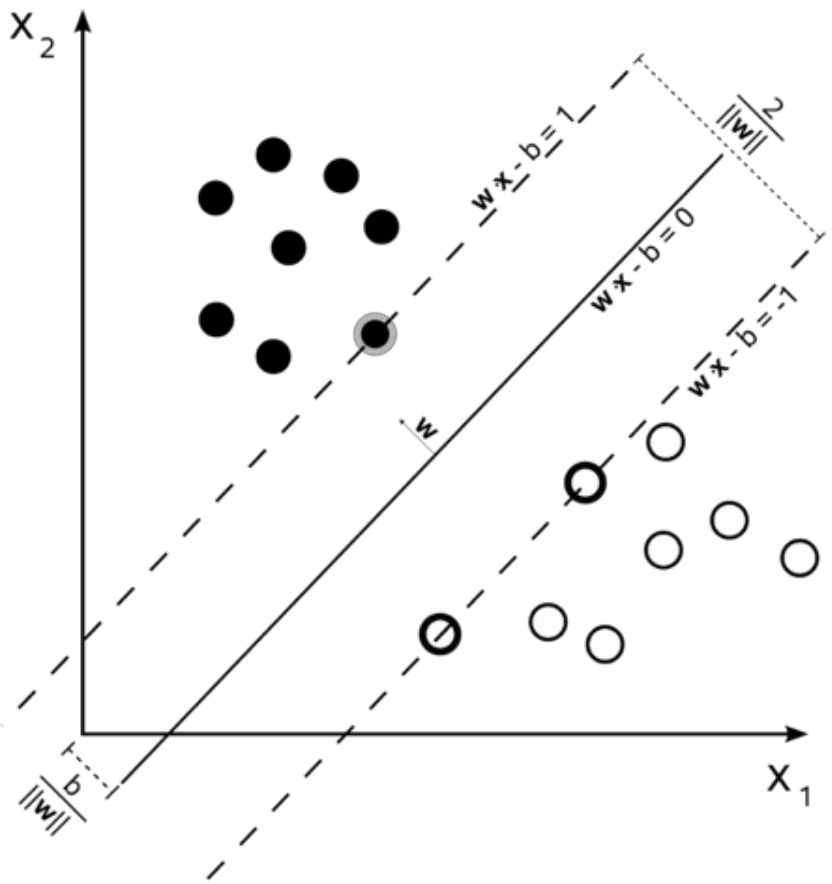
Support Vector Machines (SVM)

The Margins (Dashed Lines):

- The two dashed lines are the **margins**. They create a "safe zone" around the dividing line.
- The SVM algorithm tries to make this margin as **wide as possible**, which results in a more robust and better model.

Support Vectors (Circled Dots/Circles):

- These are the **crucial data points** that are **closest** to the dividing line and sit *right on the margin*.
- They are the "borderline" cases.
- The SVM model only uses these few, important points to define the optimal boundary; all the other dots further away are ignored.

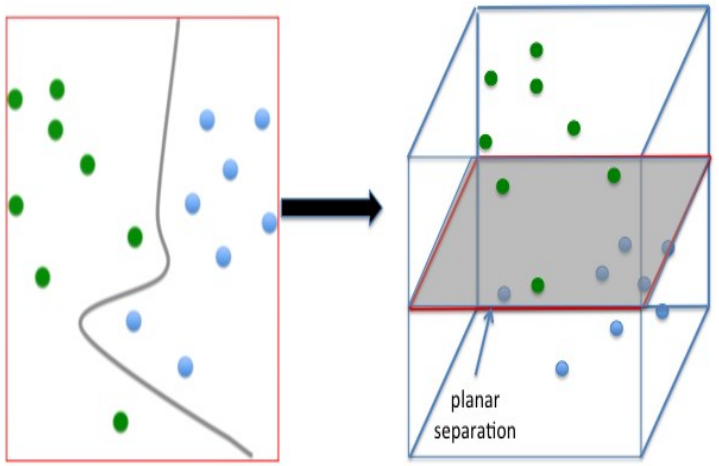


Visualizing the Algorithm

- This diagram shows how **Support Vector Machines (SVM)** solve problems that are **hard to separate** in 2D space by moving them into a higher dimension.

2D to 3D Mapping:

- Imagine you have two groups of dots (green and blue) on a flat sheet of paper (2D).
- **The Problem:** The line separating them is **curved and messy** (the wavy grey line). It's difficult to find a simple, straight rule to separate the two groups.



The Solution (3D View)

- SVM uses a mathematical trick (a **kernel function**) to lift the data off the paper and place it into a higher dimension (like a 3D cube).
- **The Result:** By moving the points, the complicated, curved separation suddenly becomes **easy and straight**.
- **Planar Separation:** In this new 3D space, you can now use a simple, flat plane (the grey square) to perfectly separate all the green dots from the blue dots.

Why Errors Aren't Always Bad

Non-separable Training Sets:

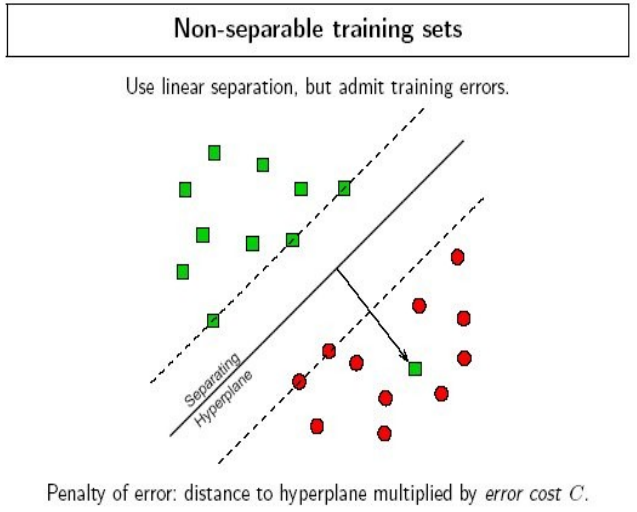
- The green squares and red circles are mixed together in the middle.
- It's impossible to draw a perfectly straight line to separate them without crossing some points.
- If you try to draw a perfectly wiggly line to get every single training point correct, your model will be too complex and won't work well on new, unseen data (this is called overfitting).

The **SVM Solution**:

- The SVM method chooses to "admit training errors.“
- It draws a simple, straight line (Separating Hyperplane) that is the best fit overall.
- This straight line makes a few mistakes (the points that are on the wrong side of the solid line, like the one green square and one red circle), but it creates a robust, wide margin (the dashed lines).

Penalty of Error (C):

- The model doesn't ignore the mistakes; it applies a penalty for them.
- The further a misclassified point is from the line, the bigger the penalty, controlled by a cost value (C).

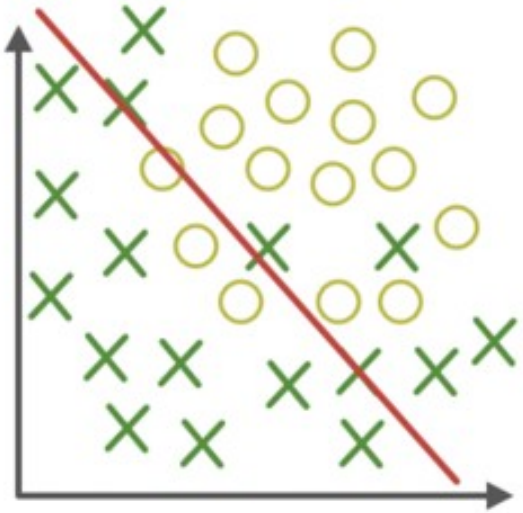


The Goal of a Model

- Imagine you have a big pile of data (like the yellow circles and green 'X's).
- You want a computer model to draw a perfect line or curve that separates the two types of data.

Underfitting: The Too-Simple Model

- **What it is:** The model is **too simple** and doesn't learn enough from the training data.
- **The Problem:** The line will miss a lot of items and be **inaccurate** for new data.



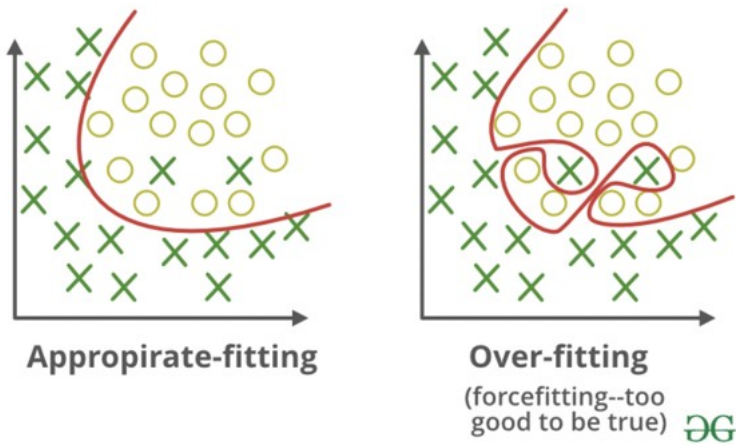
Under-fitting
(too simple to explain the variance)

Overfitting: The Too-Complex Model

- **What it is:** The model learns the training data **too well**, including all the little random **"noise"** or quirks that won't show up in new data.

The Problem:

- It's like making a rule for your laundry based on the *exact* spot every shirt landed on the floor.
- It's perfect for *this specific pile*, but the rule is so convoluted it won't work for *any* new pile of laundry. It's **too specific** and can't generalize.
- Look at the **"Over-fitting"** graph. The red line is highly complex, trying to perfectly loop around every single data point, even ones that seem out of place.
- This complexity is the problem, it won't hold up when you give it new data.



Appropriate-fitting

- This is the ideal model that learns the **main patterns** in the data without getting distracted by the "noise."
- **The Result:** It's complex enough to be **accurate** but simple enough to work well when predicting **new, unseen data**.
- Look at the **"Appropriate-fitting"** graph. The curved red line separates the groups nicely without being overly complicated.

Syracuse University
School of Information Studies

Coding

Syracuse University
School of Information Studies

The End